Lesson Activities — 1.1 Processors, Input, Output & Storage

Teacher-facing pack of in-lesson activities for OCR H446 Section 1.1 — varied, mostly zero-prep, designed for engagement and deep processing rather than exam drilling.

1.1.1 Structure and Function of the Processor

Living FDE Cycle (unplugged role-play · 15 min · groups of 8 or whole class)

Resources: 8 labelled sticky notes/lanyards (PC, MAR, MDR, CIR, CU, ALU, ACC, Memory); 4–5 instruction cards (e.g. `LDA 12`, `ADD 13`, `STA 14`, `HLT`); a row of desks as "memory" with numbered addresses. **How it runs:** 1. Assign roles: Memory sits at desks holding instruction cards at numbered addresses; the other seven students stand in a "CPU zone" wearing their register/component labels. PC holds a card reading "0". 2. Walk the fetch: PC shows their number to MAR; MAR shouts the address down the "address bus" (one-way — MAR may speak to Memory, Memory may never speak back on that channel); Memory hands the instruction card to MDR via the "data bus"; PC increments their card; MDR passes the card to CIR. 3. CU takes the card from CIR, tears/splits it (or reads it aloud) into opcode and operand, and directs the execute: loads go via MDR to ACC; `ADD` sends both values to ALU, who does the maths and hands the result to ACC. 4. Repeat for each instruction until `HLT`. Ban all shortcuts — if a student passes data along a route that doesn't exist (e.g. Memory talking directly to CIR), freeze the room and ask the class to spot the illegal move. 5. Round 2: swap roles and run a program containing a branch (`BRZ`) — the class must work out that execute writes a **new address into PC**. **Answers / expected outcomes:** Correct fetch order: PC→MAR, address bus out (unidirectional), instruction back via data bus→MDR, PC incremented, MDR→CIR. Decode = CU splits opcode/operand. Execute = ALU result→ACC, or store via MDR, or jump rewrites PC. Common errors to fish for: forgetting to increment PC, skipping MDR→CIR, letting the address bus run both ways. **Stretch:** Ask the branching PC-holder: "What just happened to the two instructions the pipeline had already prefetched?" (flushed — link to branch hazards).

Human Pipeline Laundry Line (kinesthetic demo · 10 min · 3 volunteers + whole class)

Resources: 6+ "instruction" cards; three desks labelled FETCH, DECODE, EXECUTE; a visible clock (teacher claps = one clock cycle). **How it runs:** 1. Round 1 — no pipelining: one card at a time travels through all three desks before the next card may start. Class counts the claps needed for 6 instructions (18 cycles). 2. Round 2 — pipelining: three volunteers, one per desk; on each clap every desk passes its card forward and FETCH takes a new one. Class counts again (8 cycles for 6 instructions). 3. Ask: "Did any single instruction get faster?" (No — latency unchanged; **throughput** improved.) 4. Round 3 — sabotage: make instruction 3 a branch card. When it executes and jumps, the two cards already in FETCH and DECODE must be dramatically thrown in the bin (pipeline flush). Recount the cycles. 5. Debrief: link to laundry (wash/dry/fold) and to why RISC's fixed-length instructions pipeline more easily than CISC's variable-length ones. **Answers / expected outcomes:** 6 instructions: 18 cycles unpipelined vs 8 pipelined; students articulate that pipelining overlaps fetch/decode/execute of consecutive instructions and that branches cause flushes (branch hazards), reducing the benefit. **Stretch:** "Instruction 4 needs the result of instruction 3, which hasn't executed yet — what are the options?" (data hazard: stall, or out-of-order execution/branch prediction).

Register Odd One Out (odd one out · 8 min · pairs, mini-whiteboards)

Resources: Board with four terms: **PC, MAR, MDR, ACC**. Mini-whiteboards. **How it runs:** 1. Pairs choose an odd one out and write a justification — insist "there is more than one right answer; marks are for the reasoning." 2. Collect answers via whiteboards; tally which register was picked. 3. Challenge each pair to now justify a *different* register as the odd one out. 4. Repeat with round 2: **ALU, CU, cache, ACC**; round 3: **address bus, data bus, control bus, system clock**. **Answers / expected outcomes:** Round 1 — MDR (only one holding data/instructions, not an address or result); PC (only one that self-increments / only one holding the *next* rather than current item); ACC (only one written by the ALU, not involved in fetch); MAR (only one connected to the unidirectional address bus). Round 2 — cache (not inside the processor core / not a component of FDE); ACC (a register, not a unit). Round 3 — address bus (unidirectional); clock (not a bus). **Stretch:** Students write their own four-term odd-one-out where every term is defensible, and trade with another pair.

Always–Sometimes–Never: CPU Performance (statement judgement · 10 min · pairs then whole class)

Resources: Statements on the board; mini-whiteboards showing A / S / N. Zero prep beyond the list. **How it runs:** 1. Display one statement at a time; pairs discuss 30 seconds, then show A/S/N simultaneously. 2. Pick a pair from each camp to justify; the "Sometimes" answers must state *when* it's true and *when* it's false. 3. Statements: (a) "Doubling the clock speed doubles how fast a program runs." (b) "A quad-core CPU runs a program faster than a dual-core." (c) "The address bus carries data from the CPU to memory." (d) "A cache miss means the data must be fetched from RAM." (e) "Increasing MAR width increases the amount of addressable memory." (f) "Pipelining makes each individual instruction execute faster." 4. Close by having pairs convert one "Sometimes" into an "Always" by adding the missing caveat. **Answers / expected outcomes:** (a) Sometimes — memory/I-O bottlenecks, heat throttling. (b) Sometimes — only if the task is parallelisable and the software supports it. (c) Never — it carries addresses, one-way CPU→memory. (d) Sometimes — L1 miss may hit L2/L3 before RAM. (e) Always — 2^n locations. (f) Never — throughput rises, per-instruction latency doesn't. **Stretch:** Students write one "Sometimes" statement about cache levels that would split the class.

Whiteboard Quick-Draw: Fetch From Memory (drawing from memory · 8 min · individual)

Resources: Mini-whiteboards; the knowledge organiser closed. **How it runs:** 1. Books closed. "Draw the CPU: five registers, ALU, CU, memory, and the three buses. You have three minutes." 2. Students then number arrows 1–4 showing the fetch stage only. 3. Pair-check against a projected model answer; each student writes one thing they missed on the corner of their board. 4. Erase; redraw from memory in two minutes. Compare. **Answers / expected outcomes:** Diagram must show: address bus with a one-way arrow, data and control buses two-way; fetch sequence 1 PC→MAR, 2 read signal on control bus + instruction via data bus→MDR, 3 PC+1, 4 MDR→CIR. The two classic omissions (PC increment; MDR→CIR) are the success criteria. **Stretch:** Add the decode and execute annotations for `ADD 13`, including where the ALU result lands.

1.1.2 Types of Processor

RISC ↔ CISC Decision Continuum (physical continuum · 10 min · whole class)

Resources: Masking tape or an imaginary line across the room: one wall = "definitely RISC", other = "definitely CISC". Scenario list. **How it runs:** 1. Read a scenario; students stand where they judge along the line — no fence-sitting in the exact middle without a spoken reason. 2. Interview outliers and the cluster: "What feature of the scenario put you there?" 3. Scenarios: (a) fitness tracker running two weeks on a coin cell; (b) desktop workstation running legacy x86 software; (c) drone flight controller; (d) data-centre server prioritising performance-per-watt; (e) student's own laptop. 4. After each, one student must give the *hardware* justification (instruction length, cycles per instruction, power, pipelining) before moving on. **Answers / expected outcomes:** (a) RISC — low power, simple fixed-length instructions. (b) CISC — x86 compatibility, complexity in hardware. (c) RISC — embedded, low power, real-time. (d) Defensibly RISC (ARM servers) — good discussion point. (e) Genuinely mixed — Apple Silicon is RISC; many laptops CISC — reward students who ask what the laptop is. **Stretch:** "Modern x86 chips translate CISC instructions into RISC-like micro-ops internally — does the distinction still matter? Defend a position."

Von Neumann / Harvard / Both Card Sort (card sort · 10 min · pairs)

Resources: 12 statement cards per pair (or statements on the board sorted on mini-whiteboards into three columns: VN / Harvard / Both). **How it runs:** 1. Pairs sort statements: "single shared memory for data and instructions"; "separate buses for data and instructions"; "can fetch an instruction while reading data"; "suffers a bus bottleneck"; "cheaper and simpler to build"; "used in general-purpose PCs"; "used in embedded systems and DSPs"; "follows the fetch–decode–execute cycle"; "instructions are stored in memory"; "data and instructions can be the same width"; "a program could accidentally overwrite its own instructions"; "modern CPUs use a hybrid of this at cache level". 2. Pairs compare with neighbours and argue any differences. 3. Whole-class check; dwell on the "Both" cards — students often assume everything must split two ways. 4. Fastest finishers write one extra card for each column. **Answers / expected outcomes:** VN: shared memory, bottleneck, cheaper/simpler, general-purpose PCs, can overwrite own instructions. Harvard: separate buses, simultaneous fetch+data access, embedded/DSP, different widths possible. Both: FDE cycle, instructions stored in memory, hybrid at cache level (modified Harvard — split L1 I/D cache over unified RAM). **Stretch:** Justify why the "modified Harvard" hybrid puts the split at L1 cache rather than at main memory.

CPU or GPU? Corner Sort (kinesthetic sort · 8 min · whole class)

Resources: Two corners labelled CPU and GPU, a third spot labelled "It depends". Workload list. **How it runs:** 1. Read a workload; students walk to a corner within five seconds, then justify on request. 2. Workloads: training a neural network; applying a filter to every pixel of a photo; running the OS scheduler; a chess engine evaluating deep branching game trees; mining cryptocurrency hashes; simulating weather across a grid of cells; parsing a program's source code. 3. After each round, ask one student in the majority corner to state the *general principle* (same operation on large data blocks in parallel vs sequential/branch-heavy/dependent work). 4. Finish with: "Write the rule of thumb in one sentence on your whiteboard." **Answers / expected outcomes:** GPU: neural network training, pixel filter, hashing, grid weather simulation (data parallelism — same op, huge data). CPU: OS scheduler, chess tree search (branch-heavy, dependencies), parsing (sequential). Rule: GPUs win when the same simple operation applies independently across massive data; CPUs win on sequential, branchy, dependency-laden work. **Stretch:** "Amdahl's intuition: if 20% of a task is inherently serial, what's the best possible speed-up however many cores you add?" (5×).

Just a Minute: Processor Types (speaking challenge · 10 min · groups of 4)

Resources: None — topic slips optional (RISC, CISC, GPU, multicore, parallel processing, Harvard architecture). **How it runs:** 1. In fours: one speaker, one timer, two challengers. Speaker must talk on their processor topic for 60 seconds without hesitation, repetition of the topic word, or saying anything technically false. 2. Challengers may buzz for waffle, error, or the banned word; a successful challenge steals the topic and the remaining time. 3. Whoever holds the floor at 60 seconds scores a point; the group logs any *disputed* technical claim for the teacher. 4. Rotate topics so everyone speaks; teacher adjudicates the disputed-claims list with the class. **Answers / expected outcomes:** Success = 60 seconds of accurate content, e.g. for RISC: few simple fixed-length instructions, one instruction per cycle aim, complexity pushed to the compiler, easier pipelining, lower power, ARM/mobile. The disputed-claims debrief surfaces misconceptions (e.g. "RISC is always slower" — false). **Stretch:** Ban a second word (for GPU, ban "graphics") to force deeper vocabulary (cores, data parallelism, GPGPU).

Analogy Autopsy: 1,000 Pupils vs 4 Professors (analogy critique · 8 min · think-pair-share)

Resources: Analogy on the board: "A GPU is 1,000 primary-school pupils each doing one simple sum; a CPU is 4 professors solving hard problems one after another." **How it runs:** 1. Think (1 min): individually list what the analogy gets *right*. 2. Pair (2 min): find at least two places the analogy *breaks down*. 3. Share: harvest onto a two-column board (Holds up / Breaks down). 4. Pairs propose a one-word patch to the analogy that fixes one flaw. **Answers / expected outcomes:** Holds up: many simple cores vs few powerful ones; same simple operation en masse; professors better for sequential/complex logic. Breaks down: GPU cores execute in lockstep (SIMD) — pupils can't each freelance a *different* sum; pupils don't need identical instructions, GPU cores largely do; no communication/coordination cost shown; professors (CPU cores) also run in parallel with each other; nothing represents memory bandwidth, which often limits GPUs. **Stretch:** Write a better original analogy for pipelining or cache and give it to a partner for autopsy.

1.1.3 Input, Output and Storage

Storage Shopping Task: Spend the Budget (decision task · 20 min · groups of 3)

Resources: One projected "catalogue" (no printing needed): 4 TB HDD £70 · 1 TB SATA SSD £55 · 2 TB NVMe SSD £120 · 128 GB USB flash drive £12 · 25 GB Blu-ray discs £1 each (drive £60) · LTO tape 12 TB £40 (drive £1,500) · Cloud storage £8/TB/month. Scenario cards on slides. **How it runs:** 1. Each group is an IT consultant with a client and a budget. Scenarios: (a) video-editing freelancer, £200, needs fast scratch space + archive of finished projects; (b) hospital, £3,000, must keep 60 TB of scans off-site for 8 years; (c) school, £150, distributing a 4 GB resource pack to 30 staff who mostly have no optical drives; (d) wildlife charity, £100, camera trap storage in a remote, hot, vibration-prone hide. 2. Groups pick devices from the catalogue, staying in budget, and write a two-sentence justification per choice linking a *characteristic* to a *stated requirement*. 3. Groups pitch to another group acting as the sceptical client, who must ask one "why not X instead?" question. 4. Teacher debrief: highlight pitches that compared ("whereas…") versus pitches that just listed generic pros. **Answers / expected outcomes:** (a) NVMe SSD for fast scratch (no moving parts, high transfer) + HDD for cheap-per-GB archive — buying only SSD blows capacity, only HDD too slow for editing. (b) Tape: lowest £/GB at scale, long shelf life, serial access acceptable for archive; drive cost amortised over 60 TB; cloud defensible but recurring cost ≈ £5,760/year exceeds tape long-term. (c) Cloud/network share or USB drives; Blu-ray fails "no optical drives". (d) Flash (SD/USB): durable, no moving parts (vibration kills HDDs), low power, heat-tolerant. Success criterion:

every justification pairs characteristic ↔ requirement. **Stretch:** Add a curveball mid-pitch ("the hospital now needs any scan retrievable within one minute") and make groups defend or revise (tape's serial access now fails).

Magnetic / Flash / Optical Human Sort (kinesthetic card sort · 10 min · 12–15 volunteers or pairs at desks)

Resources: Statement cards (one per student, or one set per pair): "stores data as magnetised regions on a platter"; "laser reads pits and lands"; "traps electrons in memory cells"; "no moving parts"; "wear levelling extends its life"; "read/write head on an actuator arm"; "cheapest per disc to post to a customer"; "vulnerable to scratches"; "finite number of write cycles"; "high capacity at low cost per GB"; "serial access when on tape"; "silent and shock-resistant". **How it runs:** 1. Label three zones of the room MAGNETIC / FLASH / OPTICAL. Students read their card aloud and walk to a zone. 2. Each zone audits itself: anyone who looks wrong is cross-examined by the other zones. 3. Teacher calls two deliberate wobbles: "high capacity at low cost per GB" (magnetic — but let flash argue) and "no moving parts" (flash — but optical *discs* have no moving parts; the *drive* does). 4. Zones each compose one new true statement that could only belong to them. **Answers / expected outcomes:** Magnetic: platter magnetisation, R/W head, low £/GB high capacity, tape serial access. Flash: electron trapping, no moving parts, wear levelling, finite write cycles, silent/shock-resistant. Optical: pits and lands via laser, scratches, cheap per disc. Students should verbalise *how it works* language ready for 2–3 mark "describe" answers. **Stretch:** Zone representatives must explain why their medium is non-volatile using its physical mechanism.

Spot and Fix: The Terrible Memory Essay (error hunt · 10 min · pairs)

Resources: Projected paragraph containing planted errors (below). Mini-whiteboards for corrections. **How it runs:** 1. Project: *"RAM is non-volatile main memory that holds running programs. ROM is volatile and stores the user's documents. When RAM is full, virtual memory adds extra RAM chips so more programs can run. Pages are swapped between the hard drive and ROM, and because the disk is faster than RAM this speeds the computer up. If too much swapping occurs the system enjoys disk thrashing, which improves performance. Cache is a section of RAM that is slower than main memory."* 2. Pairs find and number every error (tell them how many: 8). 3. Pairs rewrite the worst two sentences correctly on whiteboards. 4. Class-check; award a point per error plus a point per accurate fix. **Answers / expected outcomes:** Errors: (1) RAM is volatile, not non-volatile; (2) ROM is non-volatile; (3) ROM stores firmware/bootstrap, not user documents; (4) virtual memory does NOT add RAM — it uses secondary storage as if it were RAM; (5) pages swap between disk and RAM, not ROM; (6) disk is far slower than RAM, so it slows things down; (7) thrashing degrades performance, nothing to "enjoy"; (8) cache is separate fast memory on/near the CPU, faster than RAM and not part of it. **Stretch:** Pairs write their own 3-error paragraph on input/output devices for another pair to debug.

Device Doctors Jigsaw (jigsaw peer-teaching · 20 min · home groups of 4)

Resources: Knowledge organiser section per expert group; blank A5 "prescription pad" (any paper) per student. **How it runs:** 1. Home groups of four number off 1–4. Experts regroup: (1) magnetic storage, (2) flash/SSD, (3) optical + tape archive, (4) RAM/ROM/virtual memory & cloud storage. 2. Expert groups get 5 minutes to master their area and agree the three facts a novice must not leave without, plus one "killer scenario" their tech is best for. 3. Return to home groups: each expert teaches for 90 seconds, others take notes with pens down until the minute is up (listen first, write after). 4. Home groups then face a teacher scenario ("a photographer needs field storage and a 20-year archive") requiring at least two experts' knowledge combined. 5. Books closed: each student writes one sentence from an area they did NOT teach. **Answers / expected outcomes:** Accurate mini-lessons matching the knowledge organiser;

scenario answer combines flash (field: durable, fast, no moving parts) with optical or magnetic archive (cheap, long shelf life); step-5 sentences reveal whether teaching transferred. **Stretch:** Experts must field one hostile question from their home group ("but why not just use X?") without notes.

Odd One Out: Storage Line-Up (odd one out · 6 min · pairs, mini-whiteboards)

Resources: Board: **HDD · SSD · DVD · RAM**. **How it runs:** 1. Pairs pick the odd one out and justify; remind them multiple answers are defensible. 2. Harvest via whiteboards; require a *different* justification from each pair called on. 3. Round 2: **USB stick · SD card · SSD · magnetic tape**. Round 3: **barcode scanner · touchscreen · microphone · actuator**. 4. Vote on the single most elegant justification of the lesson. **Answers / expected outcomes:** Round 1 — RAM (volatile / primary not secondary storage); DVD (optical / removable / read by laser); HDD (moving parts among the modern trio); SSD (defensible: only one with wear levelling / finite writes stated on spec sheets). Round 2 — tape (magnetic not flash; serial access). Round 3 — actuator (output; the rest are input) or touchscreen (both input and output — reward that). **Stretch:** Construct a four-device line-up where the intended odd one out is a *volatility* argument, and test it on the class.